

02-614 Week 6

String Algorithms

Aidan Jan

October 1, 2026

An Algorithm to Construct Non-Compressed Suffix Trees (Tries)

When a suffix tree is built, we read the string in from left to right. Consider the string $S = \text{abbacabaa}$. For example, when we get to $i = 4$, assume we have a valid suffix tree for the first four characters (i.e., $S_4 = \text{abba}$), T_4 , encoding all the suffixes of S_4 . To get to $i = 5$, we will add onto T_4 to create T_5 .

- To do this, notice that all the suffixes of T_4 are connected by suffix links. That is, *abba* has a link to *bba* which has a link to *ba*, etc.
- To append the next character, we start from the deepest node representing the entire string so far, add a new node for the next character, follow the suffix link, and repeat the process until the root is reached.
- Update suffix links.

This algorithm constructs the Suffix Trie, and runs in quadratic time.

Ukkonen's Algorithm

Ukkonen's Algorithm creates a suffix tree (compressed) without the extra nodes. An “extra node” is a node that does not branch, and can therefore be combined into other nodes.

- Same basic structure as the previous algorithm
- Does not create unnecessary nodes

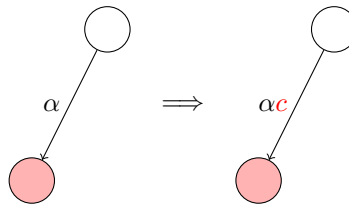
This adds on a new issue: what if we want to extend off of a middle of a node? Since a node can contain multiple characters now, there will be some cases where a node needs to be converted into a branch. This introduces multiple cases of adding nodes.

Suppose the character we want to add is c .

- **Case A:** Extending a leaf node.
- **Case B:** Extending an internal node which currently has no child with c .
- **Case C:** Extending an internal node which already has an edge for c .
- **Case D:** Extending an edge where the next character is c .
- **Case E:** Extending an edge where the next character is not c .

Case A

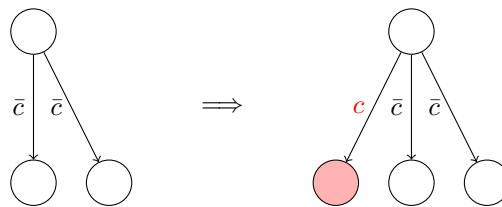
We want to extend a leaf node. (Let red nodes be leaves)



Since there is no new node, there is no new suffix link.

Case B

Extend an internal node where the node for c does not yet exist.



Since a node was added here, it receives a suffix link, pointing to whichever next node when we move up.

Case C

Extend an internal node where the node for c exists already.

- For this case, we do nothing and stop moving backwards through the suffix links.
- If the node for c exists already, then we do nothing because it already exists.
- We stop moving backwards because if we didn't change the current node, then suffix links won't be updated for any of the parent nodes either.

Case D

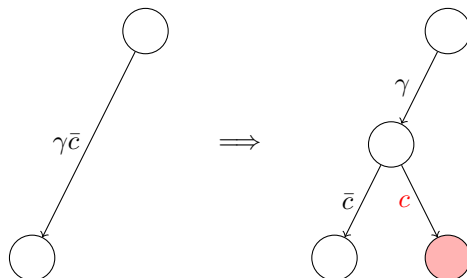
Extend an edge where the next character is c .

- Again, we do nothing for this case and stop moving backwards through the suffix links.
- If c exists in an edge, there is no reason to separate it into a new node since branching won't happen anyways.
- Moving backwards is also not necessary since because this node (or edge) did not change, the parents won't change either.

Case E

Extend an edge where the next character is not c .

- This is the tricky case; we need to break the edge and create a new node in the middle.



For suffix links for case E, when we break an edge and add a node, we lose the suffix link that points to the next location we are supposed to be at. Depending on where in the edge we added the new node, the suffix link does not exist yet. We must then take the slow way and traverse the graph downwards from the grandparent's suffix link.

- Let each move downwards through the graph be called a *hop*. Hopping down from the previous node takes $O(|\gamma|)$ time since we have to traverse $|\gamma|$ nodes.

Faster Case A

Recall that suffix trees don't actually store the characters but rather indices in the original string.

- To do case A faster, suppose a leaf node stores indices $x : y$.
- Adding the current letter gives $x : y + 1$ instead. However, $y + 1$ is always the length of the string we are currently considering.
- To improve the running time, we can instead store a pointer at the leaf nodes, basically $x : [\text{int}^* \text{ 1}]$, where 1 is a pointer to the length of the string we are currently considering.
- By doing this, we can update all the affected leaf nodes at the same time by just changing the length.

Where to Start a Phase

Let a *phase* be defined as a traversal from the deepest node going up the suffix links, updating the nodes with the current letter. Essentially, adding one letter to the tree.

- Suppose in the current phase, we are adding A to the tree as the next character, and the nodes we traverse have the following characters (in order): $AAAAANNANNNX$. The first character is the deepest leaf node, the N 's are characters that are not A that we add nodes to (i.e., cases B and E), and X is either case C or D where we stop traversal.
- By the next phase, the same string would look like $AAAAAAAAAAAX$. This is because we just added A , so every node on the suffix link frontier must be A .
- We can start the next phase at the place we stopped last time if we add the same letter again. (All the A 's now are leaves and are updated automatically with the length.)

Runtime

Note that all of the individual cases take constant time except for case E.

- Cases A - D are all just some local surgery.
- Case E is $O(\text{number of hops})$.

Theorem

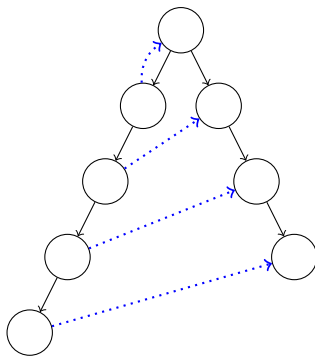
Running time is $O(|\Gamma| + \# \text{ hops})$.

For this to be linear in terms of the string, we need to show that the number of hops in total for the entire string is $O(|T|)$. We first need to prove this.

Lemma:

The number of hops is $O(|T|)$.

Proof: First, note that following a suffix link reduces the node depth by at most 1. Consider a generic branching path:



By the way suffix links are defined, links from the left branch in this case must point to those on other branch(es). Additionally, suffix links should be “parallel” in that two nodes linked by suffix links would mean their parents are also linked by suffix links. Thus, at most one node on the longer branch can point to the root; the others will be paired up as shown above.

As a consequence, if we break an edge and add a node in the middle, the most we will have to back-track would be just the number of characters the edge represents, which must be linear.

If $\Phi :=$ the current *node* depth, then

$$\begin{aligned} \Phi_{\text{final}} &\geq 0 + \sum_{\text{operation } i} (\text{hops}_i - 2) \geq \text{hops} - 4|T| \\ |T| &\geq \Phi_{\text{final}} \geq \text{hops} - 4|T| \\ \Rightarrow 5|T| &\geq \# \text{hops} \end{aligned}$$

Suffix Arrays

Suffix trees have a storage issue, which is they take $O(|\text{string}|)$ space. Normally this is fine, but not in the case of a 100-million base pair genome. A DNA sequence with an alphabet of 4 letters might take around 20 bytes per character. A **suffix array** attempts to solve this problem, by reducing the constant in the front.

Structure of Suffix Array

Let $T = \text{banana\$}$. First, we list, then sort all the suffixes.

LCP	start index	string
-	6	\$
0	5	a\$
1	3	ana\$

3	1	anana\$
0	0	banana\$
0	4	na\$
2	2	nana\$

In addition to storing the starting indexes, we also store LCP (longest common prefix) between consecutive suffixes in the sorted order.

- In the context of the suffix tree, the LCP is depth of their deepest shared node between two suffixes.
- Numbers in the suffix array are also the order suffixes are traversed in the suffix tree, if DFS'ed in alphabetical order. (assuming \$ comes before any other character)

Search in Suffix Array

Given a suffix array of T , does pattern P appear in T ?

- Note that with a suffix *tree*, it takes $O(|P|)$. We just follow the pattern down the string.
- With a suffix array, it is slightly slower: $O(|P| + \log |T|)$ time. However, a lot of times, it is good enough and better than allocating a massive space for the tree.
 - We will cover two algorithms for searching a suffix array of T . One is slightly slower at $O(|P| \log |T|)$, then we will improve it to get to the faster version.

Algorithm 1: Slow(er) Search Algorithm

We will basically run binary search on the suffix array.

- Let U be a pointer to the beginning of the suffix array, and let D be a pointer to the end of the suffix array. Let M be a pointer to the middle.
- Calculate M from U and D .
- Determine if the string we desire is alphabetically before or after M , and replace M with U or D , respectively.
- Repeat the process on one half until only one entry (or none!) are left.
- There are a total of $O(\log |T|)$ steps

The tricky part is comparing whether the string is before or after the middle, which we must do an $O(|P|)$ -time comparison.

- This gives $O(\log |T|)$ steps that take $O(|P|)$ each, making the total runtime $O(|P| \log |T|)$.

Additionally, once we narrow down to one entry, we can find all the occurrences of P in T by reading up or down the LCP array. Any string with a LCP greater than the pattern (assuming they match in the first place) would contain the entire pattern.

Algorithm 2: Fast(er) Search Algorithm

Similar to Algorithm 1, we also maintain U and D , pointers to the two ends of our current boundary. However, we will also add two more indices, u and d , which record the length of the longest prefix of P that match a prefix of U and D , respectively.

- Assume we can compute $lcp(i, j)$ for any suffixes i and j .

Suppose $u > d$.

- **Case 1:** $\text{lcp}(U, M) > u = \text{lcp}(U, P)$. If this is true, we know that P must go below M . This is because the suffix array is in sorted order, and if the first x characters of U and M match, and do not match in P , then P cannot be in the range.
- **Case 2:** $\text{lcp}(U, M) < u = \text{lcp}(U, P)$. If this is true, we know that P must go above M . This is for the opposite reason compared to case 1.
- **Case 3:** $\text{lcp}(U, M) = u = \text{lcp}(U, P)$. This case is the problem, because we cannot deduce which half P belongs in. There is a bit more logic that has to be done.
 - What we *can* do is that we no longer have to compare the like characters. If the first x characters match in U , P , and M , then we can check starting the $(x + 1)$ -th character and compare letter-by-letter. Once we find a mismatch, we can assign M to be the next U or D .

Other cases:

- If $u < d$ instead, we have the same three cases as above, but reversed.
- If $u = d$, then we compare P to M starting at $u = d$. This is because if this is the case, we know that all the suffixes in the U to D range have the same prefix.

Runtime:

- We are still running binary search ($\log(|T|)$ steps).
- The number of matches with lcp is $O(|P|)$ because each letter can be checked at most one time. (Once matched, lcp would cover it the next time.)

Finding lcp

For Algorithm 2 to work, we need to be able to find lcp between any two suffixes in constant time.

- Note that each lcp represents a section of the suffix tree used to construct the suffix array. Specifically, a set of leaf nodes.
- A lcp between two nodes could be a label on their lowest common ancestor. (Because sorted order.)
- As a result, they can be generated by propagating up the tree during the construction of the suffix array, and just stored in another array similar to lcp.
- Generating is linear time because the suffix tree has linear number of nodes. (Which is faster than the algorithm itself).